Stacks and Queues

In-Class Work

MyungJin Suh
EECE 7205

Q1 Expression evaluation using stacks

Two stacks, both with the top at the back of a vector. `values` holds integers. `ops` holds operators. The expression is a `vector<string>`. A token such as `-7` is one number, not a minus operator.

`isNumber` accepts an optional leading minus and then digits. `precedence` is 1 for `+` and `-`, 2 for `*` and `/`, 3 for `^`. An incoming operator pops the stack while the top has higher precedence, or equal precedence when the incoming operator is not `^`. That leaves `2 ^ 3 ^ 2` as $2^{(3^2)}$. `apply` does the arithmetic. Division uses the C++ quotient, then subtracts 1 when the remainder is nonzero and the signs differ, which rounds toward $-\infty$. `evaluate` walks the tokens, then empties `ops`.

```
#include <iostream>
#include <string>
#include <vector>
#include <cctype>
#include <iomanip>
using namespace std;

bool isNumber(const string &t) {
    if (t.empty()) return false;
    size_t i = 0;
    if (t[0] == '-') {
        if (t.size() == 1) return false;
        i = 1;
    }
    for (; i < t.size(); i++) {
        if (!isdigit(static_cast<unsigned char>(t[i]))) return false;
    }
    return true;
}

int precedence(const string &op) {
    if (op == "^") return 3;
    if (op == "*" || op == "/") return 2;
    if (op == "+" || op == "-") return 1;
    return 0;
}

// Equal precedence pops for + - * /. ^ stays, so 2 ^ 3 ^ 2 is 2 ^ (3 ^ 2).
bool popFirst(const string &top, const string &incoming) {
    if (incoming == "^") return precedence(top) > precedence(incoming);
    return precedence(top) >= precedence(incoming);
}

int apply(int a, const string &op, int b) {
    if (op == "+") return a + b;
    if (op == "-") return a - b;
    if (op == "*") return a * b;
    if (op == "/") {
        int q = a / b;
        if (a % b != 0 && ((a < 0) != (b < 0))) q--;
        return q;
    }
    int r = 1;
    for (int i = 0; i < b; i++) r *= a;
    return r;
}

string stackText(const vector<int> &v) {
    string s = "[";
    for (int i = 0; i < (int)v.size(); i++) {
        if (i) s += "␣";
        s += to_string(v[i]);
```

```
    }
    return s + "]";
}

string stackText(const vector<string> &v) {
    string s = "[";
    for (int i = 0; i < (int)v.size(); i++) {
        if (i) s += "␣";
        s += v[i];
    }
    return s + "]";
}

void show(const string &tok, const vector<int> &values, const vector<string> &ops,
          const string &action) {
    cout << left << setw(8) << tok
         << setw(18) << stackText(values)
         << setw(14) << stackText(ops)
         << action << endl;
}

void reduce(vector<int> &values, vector<string> &ops, bool trace) {
    int b = values.back();
    values.pop_back();
    int a = values.back();
    values.pop_back();
    string op = ops.back();
    ops.pop_back();
    int r = apply(a, op, b);
    values.push_back(r);
    if (trace)
        show("", values, ops, to_string(a) + "␣" + op + "␣" + to_string(b) + "␣=␣" + to_string(r));
}

int evaluate(const vector<string> &tokens, bool trace) {
    vector<int> values;
    vector<string> ops;

    for (const string &tok : tokens) {
        if (isNumber(tok)) {
            values.push_back(stoi(tok));
        } else {
            while (!ops.empty() && popFirst(ops.back(), tok))
                reduce(values, ops, trace);
            ops.push_back(tok);
        }
        if (trace) show(tok, values, ops, "");
    }

    if (trace) cout << endl << "remaining" << endl;
    while (!ops.empty())
        reduce(values, ops, trace);
    return values.back();
}

void run(const vector<string> &expr) {
    for (int i = 0; i < (int)expr.size(); i++) {
        if (i) cout << "␣";
        cout << expr[i];
    }
    cout << endl << endl;
    cout << left << setw(8) << "token" << setw(18) << "values" << setw(14)
         << "operators" << "action" << endl;
    int result = evaluate(expr, true);
    cout << "result:␣" << result << endl << endl;
}

int main() {
    run({"8", "+", "3", "*", "2", "^", "2", "-", "20", "/", "4"});
    run({"2", "^", "3", "^", "2"});
    run({"-7", "/", "2"});
    return 0;
}
```

(b) and (c) $8 + 3 \times 2^2 - 20/4$

Top is the right-hand end of each stack. A blank token line is a reduction caused by the next operator. `remaining` is the stack after the last token.

```
8 + 3 * 2 ^ 2 - 20 / 4

token   values            operators     action
8       [8]               []
+       [8]               [+]
3       [8 3]             [+]
*       [8 3]             [+ *]
2       [8 3 2]           [+ *]
^       [8 3 2]           [+ * ^]
2       [8 3 2 2]         [+ * ^]
        [8 3 4]           [+ *]         2 ^ 2 = 4
        [8 12]            [+]           3 * 4 = 12
        [20]              []            8 + 12 = 20
-       [20]              [-]
20      [20 20]           [-]
/       [20 20]           [- /]
4       [20 20 4]         [- /]

remaining
        [20 5]            [-]           20 / 4 = 5
        [15]              []            20 - 5 = 15
result: 15

2 ^ 3 ^ 2

token   values            operators     action
2       [2]               []
^       [2]               [^]
3       [2 3]             [^]
^       [2 3]             [^ ^]
2       [2 3 2]           [^ ^]

remaining
        [2 9]             [^]           3 ^ 2 = 9
        [512]             []            2 ^ 9 = 512
result: 512

-7 / 2

token   values            operators     action
-7      [-7]              []
/       [-7]              [/]
2       [-7 2]            [/]

remaining
        [-4]              []            -7 / 2 = -4
result: -4
```

The value is 15. $2^2 = 4$, then $3 \times 4 = 12$, then $8 + 12 = 20$. The minus has lower precedence than `^`, `*`, and `+`, so those three run before `-` is pushed. Then $20/4 = 5$ and $20 - 5 = 15$.

(d) Associativity and division

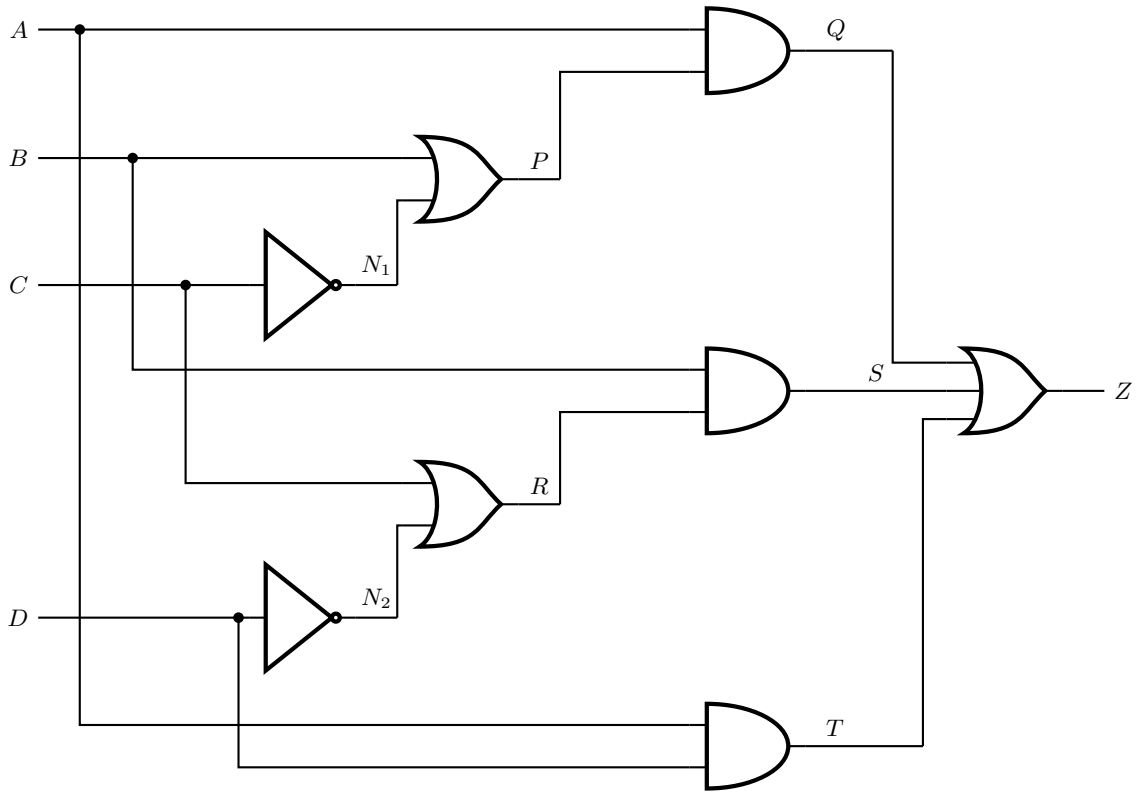
`2^3^2`: the second ^ does not pop the first, because the precedences are equal and ^ is right-associative. The stack is `[2 3 2]` over `[^ ^]`. The remaining steps are $3^2 = 9$, then $2^9 = 512$. Left associativity would have produced $(2^3)^2 = 64$.

$-7/2$: C++ gives -3. The remainder is -1, and the signs differ, so the quotient becomes -4. That is $\lfloor -7/2 \rfloor$.

Q2 Logic gate analysis

Signal	Gate	Inputs
N_1	NOT	C
N_2	NOT	D
P	OR	$B,\ N_1$
Q	AND	$A,\ P$
R	OR	$C,\ N_2$
S	AND	$B,\ R$
T	AND	$A,\ D$
Z	OR	$Q,\ S,\ T$

(a) Circuit



(b) Boolean expression

$$N_1 = C'$$
$$P = B + N_1 = B + C'$$
$$R = C + N_2 = C + D'$$
$$T = AD$$
$$Z = Q + S + T = AB + AC' + BC + BD' + AD$$

$$N_2 = D'$$
$$Q = AP = A(B + C') = AB + AC'$$
$$S = BR = B(C + D') = BC + BD'$$

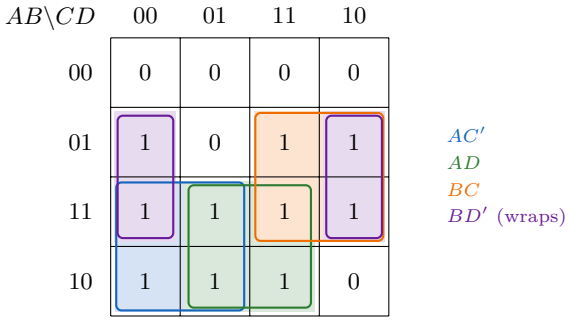
(c) Truth table

Each intermediate column is the gate in (b). $Z = Q + S + T$.

A	B	C	D	N_1	N_2	P	Q	R	S	T	Z
0	0	0	0	1	1	1	0	1	0	0	0
0	0	0	1	1	0	1	0	0	0	0	0
0	0	1	0	0	1	0	0	1	0	0	0
0	0	1	1	0	0	0	0	1	0	0	0
0	1	0	0	1	1	1	0	1	1	0	1
0	1	0	1	1	0	1	0	0	0	0	0
0	1	1	0	0	1	1	0	1	1	0	1
0	1	1	1	0	0	1	0	1	1	0	1
1	0	0	0	1	1	1	1	1	0	0	1
1	0	0	1	1	0	1	1	0	0	1	1
1	0	1	0	0	1	0	0	1	0	0	0
1	0	1	1	0	0	0	0	1	0	1	1
1	1	0	0	1	1	1	1	1	1	0	1
1	1	0	1	1	0	1	1	0	0	1	1
1	1	1	0	0	1	1	1	1	1	0	1
1	1	1	1	0	0	1	1	1	1	1	1

(d) Karnaugh map

Rows are AB and columns are CD, both in Gray code. The four quads are AC', AD, BC, and BD'. BD' uses the two outside columns.



AB is already covered, so it drops out of the sum:

$$\begin{aligned} AB &= ABC + ABC' \\ &= ABCD + ABCD' + ABC'D + ABC'D'. \end{aligned}$$

$ABCD$ and $ABCD'$ are in BC, $ABC'D$ is in AD, and $ABC'D'$ is in BD'. All four are also in AC' except the two terms that contain C. Either way,

$$Z = AC' + AD + BC + BD'.$$

Each product is a quad, and each owns a 1 that the other three miss: AC' has $m_8 = 1000$, AD has $m_{11} = 1011$, BC has $m_7 = 0111$, and BD' has $m_4 = 0100$. Four products, eight literals. This is the minimum sum of products. Factored,

$$Z = A(C' + D) + B(C + D').$$

(e) NAND only

Seven 2-input NAND gates. A complement is a NAND with the inputs tied. The inner sums are NAND by De Morgan, and the outer sum is the NAND of the two inverted products.

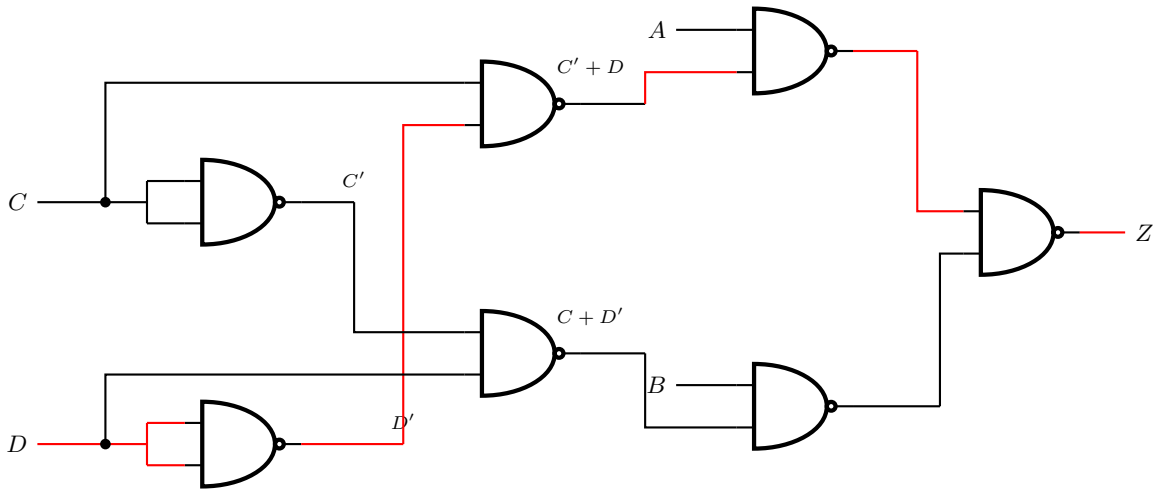
$$C' = (CC)' \qquad D' = (DD)'$$

$$C' + D = (C \cdot D')' \qquad C + D' = (C' \cdot D)'$$

$$\big(A(C' + D)\big)' = \big(A \cdot (C' + D)\big)'$$

$$\big(B(C + D')\big)' = \big(B \cdot (C + D')\big)'$$

$$Z = \Big(\big(A(C' + D)\big)' \cdot \big(B(C + D')\big)'\Big)'$$



The red wire is one longest path. The other is $C \to C' \to (C + D') \to (B(C + D'))' \to Z$.

(f) Propagation delay

Every NAND delay is 20 ns. The two paths above each pass through four gates:

$$D \to (DD)' \to (C \cdot D')' \to \big(A(C' + D)\big)' \to Z$$
$$C \to (CC)' \to (C' \cdot D)' \to \big(B(C + D')\big)' \to Z$$

No input travels through more than four gates. A and B enter at the third column, so their paths are two gates. The worst-case delay is $4 \times 20\,\text{ns} = 80\,\text{ns}$.